Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий

Кафедра вычислительной техники

ОТЧЁТ О ПРАКТИЧЕСКОЙ РАБОТЕ №6

по дисциплине
«Гибридные вычислительные системы»

«CUDA-ядро двумерной свёртки, совместимое с PyTorch»

Преподаватель _____ С. А. Тарасов
подпись, дата — инициалы, фамилия

Студент КИ__-__Б,_____ _____ _._. Афонин
номер группы, зачетной книжкой — подпись, дата — инициалы, фамилия

Студент КИ__-__Б,_____ _____ _._. Никитин
номер группы, зачетной книжкой — подпись, дата — инициалы, фамилия

Красноярск 2026

СОДЕРЖАНИЕ

1 Цель и задание 3
2 Описание реализации 4
2.1 Структура проекта 4
2.2 Алгоритм свёртки 4
2.3 Кёрнел conv2d 5
2.4 Функция запуска 8
2.5 Сборка с LibTorch 10
3 Тестирование 11
ЗАКЛЮЧЕНИЕ 13
ПРИЛОЖЕНИЕ А (обязательное) Исходный код программы 14

1 Цель и задание

Цель работы: сформировать навыки разработки CUDA-ядер, совместимых с PyTorch.

Задание:

– разработать кёрнел `hsys::nn::functional::kernels::conv2d`, который вычисляет пакетную многоканальную двумерную свёртку (кросс-корреляцию) входа X размера [N, Cin, Hin, Win] с весами W размера [Cout, Cin, k, k] с шагом s, отступом p и коэффициентом расширения d;

– используя Google Test, разработать модульные тесты для кёрнела, покрывающие декартово произведение параметров таблицы задания; корректность проверять сравнением с `torch::nn::functional::conv2d` функцией `torch::allclose` с допусками atol = rtol = 10^{-5} для float и 10^{-2} для half;

– подготовить отчёт с ключевыми фрагментами кода и ссылкой на репозиторий.

Элемент выхода свёртки вычисляется по формуле

$$Y[n, cout, h, w] = \Sigma(cin = 0..Cin-1)\ \Sigma(r = 0..k-1)\ \Sigma(q = 0..k-1)$$

$$\hat{X}[n, cin, h \cdot s + r \cdot d - p, w \cdot s + q \cdot d - p] \cdot W[cout, cin, r, q],$$

где $\hat{X}$ совпадает с X внутри изображения и равен нулю вне его (отступ). Размер выхода по каждой оси

$$Hout = \lfloor (Hin + 2p - d \cdot (k - 1) - 1) / s \rfloor + 1,$$

$$Wout = \lfloor (Win + 2p - d \cdot (k - 1) - 1) / s \rfloor + 1.$$

2 Описание реализации

2.1 Структура проекта

Проект собирается с помощью CMake и Ninja, используется стандарт C++20 и CUDA Toolkit 13.3, LibTorch подключается командой `find_package(Torch)`. Исходный код разделён на каталоги:

– core – заголовочная библиотека `hsys::nn`: кёрнел и функция запуска;

– tests – модульные тесты на Google Test.

Пространства имён повторяют структуру PyTorch: функция `hsys::nn::functional::conv2d` соответствует `torch::nn::functional::conv2d`, а кёрнел лежит во вложенном пространстве `kernels`. Параметры фильтра – аргументы шаблона, поэтому библиотека заголовочная: экземпляры кёрнела порождаются в единицах трансляции CUDA потребителя. Состав библиотеки приведён в таблице 1.

Таблица 1 – Компоненты библиотеки и тестов

Компонент	Назначение
`kernels::conv2d`	Кёрнел свёртки с тайлами в разделяемой памяти (kernels/conv2d.cuh)
`kernels::output_size`	Размер выхода по формулам задания (kernels/conv2d.cuh)
`kernels::input_tile_size`	Сторона тайла входа для тайла выхода (kernels/conv2d.cuh)
`functional::conv2d`	Проверка аргументов, выделение выхода, выбор типа и запуск кёрнела (functional/conv2d.cuh)
`conv2d_instance`	Реестр экземпляров conv2d для тестов (tests/conv2d_instances.cu)

2.2 Алгоритм свёртки

Прямая реализация, в которой нить вычисляет один выход и сама читает из глобальной памяти все $C_{in} \cdot k^2$ нужных ей элементов входа, упирается в пропускную способность памяти: соседние нити читают почти одни и те же элементы, а каждый вес читается всеми нитями. Поэтому выбран алгоритм с тайлами в разделяемой памяти.

Блок из B×B нитей (B = `block_size`, по умолчанию 16) вычисляет тайл выхода B×B для одного изображения n и группы из восьми выходных каналов. Тайл выхода зависит от квадратного участка входа со стороной

$$T = (B - 1)\cdot s + (k - 1)\cdot d + 1.$$

Для каждого входного канала блок загружает этот участок в разделяемую память, записывая нули за пределами изображения, – так реализуется отступ $\hat{X}$ без ветвлений во внутреннем цикле. Затем загружаются веса восьми каналов группы для этого входного канала, после барьера каждая нить выполняет $8\cdot k^2$ умножений с накоплением в регистрах. Элемент тайла входа используется k^2 соседними нитями и восемью каналами, вес – всеми B^2 нитями блока. Размеры тайлов для параметров задания приведены в таблице 2.

Таблица 2 – Тайлы для параметров задания (B = 16, тип float)

k, s, p, d	**Тайл входа**	**Разделяемая память, байт**	**Умножений на загруженный элемент**
3, 2, 1, 1	33×33	4644	15.9
3, 1, 1, 1	18×18	1584	46.5
7, 2, 3, 1	37×37	7044	57.0
1, 2, 0, 1	31×31	3876	2.1
3, 1, 2, 2	20×20	1888	39.1
3, 2, 2, 2	35×35	5188	14.2

Последний столбец показывает, сколько умножений с накоплением приходится на один элемент, загруженный из глобальной памяти. В прямой реализации это значение равно единице. Разделяемой памяти требуется не более 7 КиБ, поэтому на SM помещается несколько блоков.

Число пар «изображение × группа каналов» равно $N\cdot\lceil C_{out}/8\rceil$. Они распределяются по измерению z сетки, но не больше 65535 блоков: оставшиеся пары блок обрабатывает в цикле с шагом gridDim.z, поэтому размер пакета не ограничен пределом сетки.

2.3 Кёрнел conv2d

Реализация кёрнела приведена в листинге 1. Сигнатура совпадает с заданной: тензоры передаются через `torch::PackedTensorAccessor32` –

объект с указателем, размерами и шагами тензора, который индексируется на устройстве как четырёхмерный массив.

Листинг 1 – Кёрнел conv2d (core/include/hsys/nn/functional/kernels/conv2d.cuh)

```
template <class AtomT, long filter_size, long stride = 1, long padding = 0, long
     dilation = 1,
          long block_size = 16>
__global__ void __launch_bounds__(block_size * block_size) conv2d(view_4d_t<AtomT>
     output, const view_4d_t<AtomT> input,
                       const view_4d_t<AtomT> weights) {
    static_assert(filter_size > 0 && stride > 0 && padding >= 0 && dilation > 0,
                  "Invalid convolution parameters");
    static_assert(block_size > 0 && block_size * block_size <= 1024,
                  "Block must contain between 1 and 1024 threads");


    using AccT = at::acc_type<AtomT, /*is_cuda=*/true>;


    constexpr int kTile = input_tile_size(filter_size, stride, dilation, block_size);
    constexpr int kFilterArea = filter_size * filter_size;
    constexpr int kThreads = block_size * block_size;


    static_assert(sizeof(AccT) * (kTile * kTile + kOutChannelsPerBlock * kFilterArea)
     <=
                      48 * 1024,
                  "Tiles do not fit into 48 KiB of static shared memory");


    __shared__ AccT tile[kTile][kTile];
    __shared__ AccT filters[kOutChannelsPerBlock][filter_size][filter_size];


    const int tx = threadIdx.x;
    const int ty = threadIdx.y;
    const int tid = ty * block_size + tx;


    const int h_out = blockIdx.y * block_size + ty;
    const int w_out = blockIdx.x * block_size + tx;


    // Левый верхний угол тайла во входе; отрицателен в первом тайле при padding > 0.
    const int h_origin = static_cast<int>(blockIdx.y * block_size * stride) - padding;
    const int w_origin = static_cast<int>(blockIdx.x * block_size * stride) - padding;


    const int in_channels = input.size(1);
    const int out_channels = output.size(1);
    const int channel_groups = (out_channels + kOutChannelsPerBlock - 1) /
     kOutChannelsPerBlock;
    const int work_items = output.size(0) * channel_groups;


    // Условие цикла одинаково для всех нитей блока, поэтому барьеры внутри него
     безопасны.
    for (int item = blockIdx.z; item < work_items; item += gridDim.z) {
        const int n = item / channel_groups;
        const int c_first = item % channel_groups * kOutChannelsPerBlock;


        AccT acc[kOutChannelsPerBlock] = {};


        for (int c_in = 0; c_in < in_channels; ++c_in) {
            // Соседние нити загружают соседние элементы строки тайла: обращения
     объединяются.
            // Вне изображения записывается ноль — это и есть X^ из формулы (2).
            for (int i = tid; i < kTile * kTile; i += kThreads) {
```

```
                const int h = h_origin + i / kTile;
                const int w = w_origin + i % kTile;
                const bool inside = 0 <= h && h < input.size(2) && 0 <= w && w <
    input.size(3);
                tile[i / kTile][i % kTile] =
                    inside ? static_cast<AccT>(input[n][c_in][h][w]) : AccT{};
            }

            // Веса отсутствующих каналов последней группы заменяются нулями.
            for (int i = tid; i < kOutChannelsPerBlock * kFilterArea; i += kThreads) {
                const int c = i / kFilterArea;
                const int r = i % kFilterArea / filter_size;
                const int q = i % filter_size;
                filters[c][r][q] = c_first + c < out_channels
                                       ? static_cast<AccT>(weights[c_first +
    c][c_in][r][q])
                                       : AccT{};
            }
            __syncthreads();

            // Все нити варпа читают один и тот же вес — широковещательное чтение без
    конфликтов.
#pragma unroll
            for (int r = 0; r < filter_size; ++r) {
#pragma unroll
                for (int q = 0; q < filter_size; ++q) {
                    const AccT x = tile[ty * stride + r * dilation][tx * stride + q *
    dilation];
#pragma unroll
                    for (int c = 0; c < kOutChannelsPerBlock; ++c) {
                        acc[c] += x * filters[c][r][q];
                    }
                }
            }
            __syncthreads();
        }

        // Нити за пределами выхода участвовали только в загрузке тайла.
        if (h_out < output.size(2) && w_out < output.size(3)) {
#pragma unroll
            for (int c = 0; c < kOutChannelsPerBlock; ++c) {
                if (c_first + c < out_channels) {
                    output[n][c_first + c][h_out][w_out] = static_cast<AtomT>(acc[c]);
                }
            }
        }
    }
}

}  // namespace hsys::nn::functional::kernels
```

Ключевые решения:

– `filter_size`, `stride`, `padding`, `dilation` и `block_size` – константы времени компиляции: циклы по r, q и каналам группы разворачиваются (`#pragma unroll`), смещения в тайле `ty * stride + r * dilation` вычисляются при компиляции для каждой пары r, q, а массив `acc` целиком размещается в регистрах;

– half и bfloat16 накапливаются во float (at::acc_type<AtomT, true>), в разделяемой памяти хранится уже преобразованное значение, поэтому во внутреннем цикле нет преобразований типа;

– нити, чей выход лежит за пределами изображения, не завершаются досрочно: они участвуют в загрузке тайла и проходят оба барьера __syncthreads();

– соседние нити загружают соседние элементы строки тайла, поэтому обращения варпа к глобальной памяти объединяются; при чтении весов все нити варпа обращаются к одному адресу – это широковещательное чтение без конфликтов банков;

– __launch_bounds__(block_size * block_size) ограничивает число регистров: без него при B = 32 (1024 нити) компилятор отводил нити больше 64 регистров и запуск завершался ошибкой cudaErrorLaunchOutOfResources;

– output_size возвращает ноль при отрицательном числителе формулы размера: целочисленное деление в C++ округляет к нулю, и выражение (−1)/2 + 1 дало бы размер 1 вместо 0.

2.4 Функция запуска

Функция hsys::nn::functional::conv2d (листинг 2) связывает кёрнел с C++ API PyTorch. Она проверяет аргументы макросом TORCH_CHECK (исключение c10::Error), вычисляет размер выхода, выделяет выходной тензор с теми же параметрами, что и вход, и выбирает экземпляр кёрнела по типу элементов макросом AT_DISPATCH_FLOATING_TYPES_AND2 (float, double, half, bfloat16).

Листинг 2 – Функция запуска (core/include/hsys/nn/functional/conv2d.cuh)

```
template <long filter_size, long stride = 1, long padding = 0, long dilation = 1,
          long block_size = 16>
torch::Tensor conv2d(const torch::Tensor& input, const torch::Tensor& weights) {
    TORCH_CHECK(input.is_cuda() && weights.is_cuda(),
                "hsys::conv2d: tensors must be on a CUDA device, got ",
     input.device(), " and ",
                weights.device());
```

```
    TORCH_CHECK(input.device() == weights.device(),
                "hsys::conv2d: tensors must be on the same device, got ",
     input.device(),
                " and ", weights.device());
    TORCH_CHECK(input.scalar_type() == weights.scalar_type(),
                "hsys::conv2d: input and weights must have the same dtype, got ",
                input.scalar_type(), " and ", weights.scalar_type());
    TORCH_CHECK(input.dim() == 4, "hsys::conv2d: expected input [N, C_in, H_in, W_in],
     got ",
                input.sizes());
    TORCH_CHECK(weights.dim() == 4, "hsys::conv2d: expected weights [C_out, C_in, k,
     k], got ",
                weights.sizes());
    TORCH_CHECK(weights.size(1) == input.size(1), "hsys::conv2d: weights ",
     weights.sizes(),
                " expect ", weights.size(1), " input channels, got input ",
     input.sizes());
    TORCH_CHECK(weights.size(2) == filter_size && weights.size(3) == filter_size,
                "hsys::conv2d: kernel is compiled for a ", filter_size, "x",
     filter_size,
                " filter, got weights ", weights.sizes());


    const std::int64_t h_out =
        kernels::output_size(input.size(2), filter_size, stride, padding, dilation);
    const std::int64_t w_out =
        kernels::output_size(input.size(3), filter_size, stride, padding, dilation);
    TORCH_CHECK(h_out > 0 && w_out > 0, "hsys::conv2d: input ", input.sizes(),
                " is too small for a ", filter_size, "x", filter_size, " filter with
     padding ",
                padding, " and dilation ", dilation);


    const c10::cuda::CUDAGuard device_guard(input.device());


    torch::Tensor output =
        torch::empty({input.size(0), weights.size(0), h_out, w_out}, input.options());
    if (output.numel() == 0) {
        return output;  // Пустой пакет или C_out = 0: запуск с пустой сеткой
     недопустим.
}
```

Работа с CUDA API через PyTorch:

– `c10::cuda::CUDAGuard` делает текущим устройство входного тензора, если их несколько;

– кёрнел ставится в текущий поток PyTorch `at::cuda::getCurrentCUDAStream()`, поэтому он корректно упорядочен с другими операциями над тензорами и уважает `CUDAStreamGuard` вызывающей стороны;

– ошибка конфигурации запуска проверяется `C10_CUDA_KERNEL_LAUNCH_CHECK()`;

– `packed_accessor32` сам проверяет, что число элементов тензора меньше 2^{31}, а шаги тензора аксессор учитывает, поэтому транспонированные тензоры, срезы и формат `ChannelsLast` обрабатываются без копирования.

2.5 Сборка с LibTorch

LibTorch ищется по `Torch_DIR` или `CMAKE_PREFIX_PATH`; если они не заданы, путь запрашивается у пакета `torch` текущего интерпретатора Python (`torch.utils.cmake_prefix_path`). Флаги компиляции кода с заголовками PyTorch приведены в листинге 3.

Листинг 3 – Флаги библиотеки hsys::nn (core/CMakeLists.txt)

```
set(HSYS_TORCH_NVCC_FLAGS
    -D__CUDA_NO_HALF_OPERATORS__
    -D__CUDA_NO_HALF_CONVERSIONS__
    -D__CUDA_NO_BFLOAT16_CONVERSIONS__
    -D__CUDA_NO_HALF2_OPERATORS__
    --expt-relaxed-constexpr
)
target_compile_options(hsys_nn INTERFACE
    "$<$<COMPILE_LANGUAGE:CUDA>:${HSYS_TORCH_NVCC_FLAGS}>"
)


if(MSVC)
  # Исходники в UTF-8 (комментарии на русском); остальные предупреждения — из
     заголовков
  # LibTorch, их подавляет и cpp_extension.
  set(HSYS_MSVC_FLAGS /utf-8 /EHsc /wd4819 /wd4251 /wd4244 /wd4267 /wd4275 /wd4018
     /wd4190
                      /wd4624 /wd4067 /wd4068)
  string(REPLACE ";" "," HSYS_MSVC_FLAGS_CUDA "${HSYS_MSVC_FLAGS}")
  target_compile_options(hsys_nn INTERFACE
      "$<$<COMPILE_LANGUAGE:CXX>:${HSYS_MSVC_FLAGS}>"
      "$<$<COMPILE_LANGUAGE:CUDA>:-Xcompiler=${HSYS_MSVC_FLAGS_CUDA}>"
  )
  # Компоновщик MSVC отбрасывает torch_cuda.dll, если на неё нет прямых ссылок; тогда
  # не регистрируются CUDA-реализации операторов и torch::cuda::is_available() ==
     false.
  target_link_options(hsys_nn INTERFACE "/INCLUDE:?warp_size@cuda@at@@YAHXZ")
```

Макросы `__CUDA_NO_HALF_OPERATORS__` и аналогичные отключают операторы `cuda_fp16.h`, конфликтующие с операторами `c10::Half`, – с этими же флагами `torch.utils.cpp_extension` компилирует расширения PyTorch. Ключ компоновщика `/INCLUDE` не даёт MSVC отбросить `torch_cuda.dll`, на которую нет прямых ссылок: без неё не регистрируются CUDA-реализации операторов PyTorch. Библиотеки LibTorch копируются к исполняемому файлу тестов после сборки.

Архитектура GPU (`sm_86`) задаётся до команды `project()`. В CMakeLists.txt предыдущих работ она задавалась после неё и не действовала: `project()` уже подставлял архитектуру nvcc по умолчанию (`sm_75`), и код выполнялся на sm_86 за счёт двоичной совместимости.

3 Тестирование

Модульные тесты разработаны с использованием Google Test (листинг А.8). Параметры фильтра – аргументы шаблона, а тесты параметризуются значениями во время выполнения, поэтому каждый проверяемый набор параметров порождается заранее в файле `conv2d_instances.cu` (листинг А.7) и находится по значению функцией `conv2d_instance`. Это отделяет код CUDA от тестов: файл тестов компилируется обычным компилятором C++ и использует полный API `torch/torch.h`.

Эталон – `torch::nn::functional::conv2d` над теми же значениями, вычисленный в double и приведённый к типу входа (листинг 4). Так погрешность эталона не складывается с погрешностью кёрнела, а режим TF32 cuDNN, включённый в PyTorch по умолчанию на GPU Ampere, не влияет на сравнение. Вход заполняется значениями из [−1, 1], веса – из [−1/√fan_in, 1/√fan_in], где fan_in = Cin·k², как при инициализации `torch::nn::Conv2d`: при любом числе каналов выходы имеют порядок единицы, и относительный допуск остаётся осмысленным.

Листинг 4 – Сравнение с эталоном (tests/src/test_conv2d.cpp)

```
torch::Tensor reference(const torch::Tensor& input, const torch::Tensor& weights,
                        const FilterParams& params) {
    return F::conv2d(input.to(torch::kDouble), weights.to(torch::kDouble),
     conv_options(params))
        .to(input.scalar_type());
}


::testing::AssertionResult matches_reference(const torch::Tensor& input,
                                             const torch::Tensor& weights,
                                             const FilterParams& params) {
    const torch::Tensor actual = conv2d_instance(params)(input, weights);
    const torch::Tensor expected = reference(input, weights, params);


    if (actual.sizes() != expected.sizes()) {
        return ::testing::AssertionFailure()
               << "shape " << actual.sizes() << ", expected " << expected.sizes();
    }
    if (actual.scalar_type() != input.scalar_type()) {
        return ::testing::AssertionFailure() << "dtype " << actual.scalar_type();
    }


    const auto [rtol, atol] = tolerance(input.scalar_type());
    if (torch::allclose(actual, expected, rtol, atol)) {
        return ::testing::AssertionSuccess();
    }
```

```
    const double max_error =
        (actual.to(torch::kDouble) -
     expected.to(torch::kDouble)).abs().max().item<double>();
    return ::testing::AssertionFailure() << "max |actual - expected| = " << max_error;
}
```

Наборы тестов перечислены в таблице 3.

Таблица 3 – Наборы модульных тестов

Набор	Что проверяет	Тестов
`Conv2dTableTest`	Декартово произведение таблицы задания (6 наборов параметров × 9 размеров) для float, half и bfloat16, пакет из двух изображений	162
`Conv2dEdgeTest`	Размеры, не кратные тайлу и группе каналов; выход 1×1; фильтр шире изображения; Cout = 1, 3, 9; блоки 8×8 и 32×32; чётный фильтр, шаг больше фильтра, отступ больше половины фильтра; float, double, half	57
`Conv2dExactTest`	Точное совпадение на целых значениях из [−2, 2] в double; единичный импульс воспроизводит фильтр	12
`Conv2dLayoutTest`	Транспонированный вход, ChannelsLast, срезы с шагом, непрерывность выхода	4
`Conv2dLaunchTest`	Цикл по gridDim.z (70000 пар), пустой пакет, пользовательский поток, детерминированность и неизменность входа, размер выхода против PyTorch	5
`Conv2dErrorTest`	Тензоры на CPU, разные и неподдерживаемые типы, неверная размерность, несовпадение каналов и размера фильтра, слишком маленький вход	7
`Conv2dInstancesTest`	Запрос нескомпилированного экземпляра	1

Допуски `torch::allclose` соответствуют заданию: rtol = atol = 10^{-5} для float и 10^{-2} для half и bfloat16; для double – 10^{-12}. Тест на целых значениях проверяет точное равенство (`torch::equal`): суммы небольших целых в double вычисляются без округления, поэтому любой пропущенный, лишний или сдвинутый элемент отступа меняет результат, а не теряется в допуске. При несовпадении тест выводит максимальное отклонение от эталона.

[Вставить результат запуска ctest после сборки с LibTorch: число тестов, время выполнения, максимальные отклонения для float и half.]

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы получены навыки разработки CUDA-ядер, совместимых с PyTorch.

Разработан кёрнел `hsys::nn::functional::kernels::conv2d`, вычисляющий пакетную многоканальную свёртку с шагом, отступом и расширением. Блок нитей загружает тайл входа и веса группы из восьми выходных каналов в разделяемую память, параметры фильтра заданы на этапе компиляции, что позволяет развернуть циклы. Функция запуска использует C++ API PyTorch: проверку аргументов `TORCH_CHECK`, выбор типа `AT_DISPATCH`, текущий поток и устройство PyTorch, и поддерживает float, double, half и bfloat16 для тензоров с произвольными шагами.

Разработано 248 модульных тестов на Google Test, которые сравнивают результат с `torch::nn::functional::conv2d` для всех параметров таблицы задания, граничных случаев, раскладок памяти и некорректных аргументов.

Полная реализация доступна в репозитории: https://github.com/nameofthelove/gvs-afonin-nikitin/tree/work6.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программы

Листинг А.1 – Файл `CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.24)

if(POLICY CMP0135)
  cmake_policy(SET CMP0135 NEW)
endif()

# Архитектура GPU (Ampere: RTX 3050 Ti -> sm_86); переопределяется через -
     DCMAKE_CUDA_ARCHITECTURES.
# Задаётся до project(): иначе project() подставляет архитектуру по умолчанию nvcc
     (sm_75).
if(NOT DEFINED CMAKE_CUDA_ARCHITECTURES)
  set(CMAKE_CUDA_ARCHITECTURES 86)
endif()

project(work6 LANGUAGES CXX CUDA)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CUDA_STANDARD 20)
set(CMAKE_CUDA_STANDARD_REQUIRED ON)

# Сборки LibTorch для Windows бывают только Release (/MD): Debug несовместим по CRT.
if(NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)
  set(CMAKE_BUILD_TYPE Release CACHE STRING "Build type" FORCE)
endif()

set(TORCH_CUDA_ARCH_LIST "8.6" CACHE STRING "GPU architectures for the Torch CMake
     config")

find_package(CUDAToolkit REQUIRED)

# LibTorch: Torch_DIR или CMAKE_PREFIX_PATH, иначе — пакет torch из текущего Python
# (torch.utils.cmake_prefix_path указывает на site-packages/torch/share/cmake).
if(NOT DEFINED Torch_DIR AND NOT DEFINED ENV{Torch_DIR})
  find_package(Python3 COMPONENTS Interpreter QUIET)
  if(Python3_FOUND)
    execute_process(
      COMMAND ${Python3_EXECUTABLE} -c "import torch;
     print(torch.utils.cmake_prefix_path)"
      OUTPUT_VARIABLE TORCH_CMAKE_PREFIX
      OUTPUT_STRIP_TRAILING_WHITESPACE
      RESULT_VARIABLE TORCH_PYTHON_RESULT
      ERROR_QUIET
    )
    if(TORCH_PYTHON_RESULT EQUAL 0)
      list(APPEND CMAKE_PREFIX_PATH "${TORCH_CMAKE_PREFIX}")
    endif()
  endif()
endif()
find_package(Torch REQUIRED)

include(FetchContent)
```

```
# Google Test
FetchContent_Declare(
  googletest
  URL https://github.com/google/googletest/archive/refs/tags/v1.14.0.zip
)
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
set(INSTALL_GTEST OFF CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googletest)


enable_testing()


add_subdirectory(core)
add_subdirectory(tests)
```

Листинг А.2 – Файл `core/CMakeLists.txt`

```
# Библиотека заголовочная: кёрнел и функция запуска — шаблоны с параметрами фильтра
     времени
# компиляции, экземпляры порождаются в единицах трансляции CUDA потребителя.
add_library(hsys_nn INTERFACE)
add_library(hsys::nn ALIAS hsys_nn)


target_include_directories(hsys_nn INTERFACE
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)


# Цель torch уже переносит на потребителя каталоги заголовков и TORCH_CXX_FLAGS.
target_link_libraries(hsys_nn INTERFACE
    "${TORCH_LIBRARIES}"
    CUDA::cudart
)


# Флаги, с которыми torch.utils.cpp_extension компилирует расширения: c10::Half и
# c10::BFloat16 определяют собственные операторы и преобразования, встроенные
     операторы
# cuda_fp16.h / cuda_bf16.h с ними конфликтуют.
set(HSYS_TORCH_NVCC_FLAGS
    -D__CUDA_NO_HALF_OPERATORS__
    -D__CUDA_NO_HALF_CONVERSIONS__
    -D__CUDA_NO_BFLOAT16_CONVERSIONS__
    -D__CUDA_NO_HALF2_OPERATORS__
    --expt-relaxed-constexpr
)
target_compile_options(hsys_nn INTERFACE
    "$<$<COMPILE_LANGUAGE:CUDA>:${HSYS_TORCH_NVCC_FLAGS}>"
)


if(MSVC)
  # Исходники в UTF-8 (комментарии на русском); остальные предупреждения — из
     заголовков
  # LibTorch, их подавляет и cpp_extension.
  set(HSYS_MSVC_FLAGS /utf-8 /EHsc /wd4819 /wd4251 /wd4244 /wd4267 /wd4275 /wd4018
     /wd4190
                      /wd4624 /wd4067 /wd4068)
  string(REPLACE ";" "," HSYS_MSVC_FLAGS_CUDA "${HSYS_MSVC_FLAGS}")
  target_compile_options(hsys_nn INTERFACE
      "$<$<COMPILE_LANGUAGE:CXX>:${HSYS_MSVC_FLAGS}>"
      "$<$<COMPILE_LANGUAGE:CUDA>:-Xcompiler=${HSYS_MSVC_FLAGS_CUDA}>"
  )
  # Компоновщик MSVC отбрасывает torch_cuda.dll, если на неё нет прямых ссылок; тогда
  # не регистрируются CUDA-реализации операторов и torch::cuda::is_available() ==
     false.
```

```
  target_link_options(hsys_nn INTERFACE "/INCLUDE:?warp_size@cuda@at@@YAHXZ")
endif()
```

Листинг А.3 – Файл

core/include/hsys/nn/functional/kernels/conv2d.cuh

```
#pragma once

#include <ATen/AccumulateType.h>
#include <torch/types.h>

namespace hsys::nn::functional::kernels {

template <class T>
using view_4d_t = torch::PackedTensorAccessor32<T, 4, torch::RestrictPtrTraits>;

// Число выходных каналов, которые блок вычисляет над одним загруженным тайлом входа:
// тайл читается из глобальной памяти один раз на группу каналов, а не на каждый
    канал.
inline constexpr long kOutChannelsPerBlock = 8;

// Размер выхода по одной оси, формулы (3)–(4) задания. При отрицательном числителе
    выход
// пуст: целочисленное деление в C++ округляет к нулю, а не вниз, и дало бы размер 1.
__host__ __device__ constexpr long output_size(long input_size, long filter_size, long
    stride,
                                               long padding, long dilation) {
    const long span = input_size + 2 * padding - dilation * (filter_size - 1) - 1;
    return span < 0 ? 0 : span / stride + 1;
}

// Сторона тайла входа для тайла выхода block_size x block_size: крайний выход тайла
    читает
// вход со смещением (block_size - 1) * stride + (filter_size - 1) * dilation от
    первого.
__host__ __device__ constexpr long input_tile_size(long filter_size, long stride, long
    dilation,
                                                   long block_size) {
    return (block_size - 1) * stride + (filter_size - 1) * dilation + 1;
}

// Пакетная многоканальная свёртка (кросс-корреляция), формулы (1)–(2) задания.
//
// Блок block_size x block_size нитей вычисляет тайл выхода того же размера для
    изображения n
// и группы из kOutChannelsPerBlock выходных каналов; нить отвечает за один пиксель
    тайла во
// всех каналах группы и держит суммы в регистрах. Для каждого входного канала блок
    загружает
// в разделяемую память тайл входа (с нулями вместо отступа) и веса группы, после чего
    каждый
// элемент тайла переиспользуется filter_size^2 соседними нитями и
    kOutChannelsPerBlock каналами.
// Параметры фильтра — константы времени компиляции, поэтому циклы по фильтру
    развёрнуты,
// а индексы в тайле вычисляются без умножений во время выполнения.
//
// Ожидается blockDim = (block_size, block_size), gridDim = (ceil(W_out / block_size),
// ceil(H_out / block_size), Z) при любом Z >= 1: пары (n, группа каналов) обходятся с
    шагом
// gridDim.z. Значения накапливаются в at::acc_type (float для half и bfloat16).
// __launch_bounds__ ограничивает число регистров так, чтобы блок из block_size^2
    нитей
// помещался на SM: без него при block_size = 32 запуск завершается ошибкой
```

```
// cudaErrorLaunchOutOfResources.
template <class AtomT, long filter_size, long stride = 1, long padding = 0, long
     dilation = 1,
          long block_size = 16>
__global__ void __launch_bounds__(block_size * block_size) conv2d(view_4d_t<AtomT>
     output, const view_4d_t<AtomT> input,
                       const view_4d_t<AtomT> weights) {
    static_assert(filter_size > 0 && stride > 0 && padding >= 0 && dilation > 0,
                  "Invalid convolution parameters");
    static_assert(block_size > 0 && block_size * block_size <= 1024,
                  "Block must contain between 1 and 1024 threads");


    using AccT = at::acc_type<AtomT, /*is_cuda=*/true>;


    constexpr int kTile = input_tile_size(filter_size, stride, dilation, block_size);
    constexpr int kFilterArea = filter_size * filter_size;
    constexpr int kThreads = block_size * block_size;


    static_assert(sizeof(AccT) * (kTile * kTile + kOutChannelsPerBlock * kFilterArea)
     <=
                       48 * 1024,
                  "Tiles do not fit into 48 KiB of static shared memory");


    __shared__ AccT tile[kTile][kTile];
    __shared__ AccT filters[kOutChannelsPerBlock][filter_size][filter_size];


    const int tx = threadIdx.x;
    const int ty = threadIdx.y;
    const int tid = ty * block_size + tx;


    const int h_out = blockIdx.y * block_size + ty;
    const int w_out = blockIdx.x * block_size + tx;


    // Левый верхний угол тайла во входе; отрицателен в первом тайле при padding > 0.
    const int h_origin = static_cast<int>(blockIdx.y * block_size * stride) - padding;
    const int w_origin = static_cast<int>(blockIdx.x * block_size * stride) - padding;


    const int in_channels = input.size(1);
    const int out_channels = output.size(1);
    const int channel_groups = (out_channels + kOutChannelsPerBlock - 1) /
     kOutChannelsPerBlock;
    const int work_items = output.size(0) * channel_groups;


    // Условие цикла одинаково для всех нитей блока, поэтому барьеры внутри него
     безопасны.
    for (int item = blockIdx.z; item < work_items; item += gridDim.z) {
        const int n = item / channel_groups;
        const int c_first = item % channel_groups * kOutChannelsPerBlock;


        AccT acc[kOutChannelsPerBlock] = {};


        for (int c_in = 0; c_in < in_channels; ++c_in) {
            // Соседние нити загружают соседние элементы строки тайла: обращения
     объединяются.
            // Вне изображения записывается ноль — это и есть X^ из формулы (2).
            for (int i = tid; i < kTile * kTile; i += kThreads) {
                const int h = h_origin + i / kTile;
                const int w = w_origin + i % kTile;
                const bool inside = 0 <= h && h < input.size(2) && 0 <= w && w <
     input.size(3);
                tile[i / kTile][i % kTile] =
                    inside ? static_cast<AccT>(input[n][c_in][h][w]) : AccT{};
            }
```

```
            // Веса отсутствующих каналов последней группы заменяются нулями.
            for (int i = tid; i < kOutChannelsPerBlock * kFilterArea; i += kThreads) {
                const int c = i / kFilterArea;
                const int r = i % kFilterArea / filter_size;
                const int q = i % filter_size;
                filters[c][r][q] = c_first + c < out_channels
                                       ? static_cast<AccT>(weights[c_first +
    c][c_in][r][q])
                                       : AccT{};
            }
            __syncthreads();


            // Все нити варпа читают один и тот же вес — широковещательное чтение без
    конфликтов.
#pragma unroll
            for (int r = 0; r < filter_size; ++r) {
#pragma unroll
                for (int q = 0; q < filter_size; ++q) {
                    const AccT x = tile[ty * stride + r * dilation][tx * stride + q *
    dilation];
#pragma unroll
                    for (int c = 0; c < kOutChannelsPerBlock; ++c) {
                        acc[c] += x * filters[c][r][q];
                    }
                }
            }
            __syncthreads();
        }


        // Нити за пределами выхода участвовали только в загрузке тайла.
        if (h_out < output.size(2) && w_out < output.size(3)) {
#pragma unroll
            for (int c = 0; c < kOutChannelsPerBlock; ++c) {
                if (c_first + c < out_channels) {
                    output[n][c_first + c][h_out][w_out] = static_cast<AtomT>(acc[c]);
                }
            }
        }
    }
}


}  // namespace hsys::nn::functional::kernels
```

Листинг А.4 – Файл `core/include/hsys/nn/functional/conv2d.cuh`

```
#pragma once

#include <ATen/Dispatch.h>
#include <ATen/cuda/CUDAContext.h>
#include <c10/cuda/CUDAException.h>
#include <c10/cuda/CUDAGuard.h>
#include <torch/types.h>


#include <algorithm>
#include <cstdint>


#include "hsys/nn/functional/kernels/conv2d.cuh"


namespace hsys::nn::functional {


// Предельное значение gridDim.y и gridDim.z.
inline constexpr std::int64_t kMaxGridYZ = 65535;
```

```
constexpr std::int64_t ceil_div(std::int64_t value, std::int64_t divisor) {
    return (value + divisor - 1) / divisor;
}


// Свёртка input [N, C_in, H_in, W_in] с весами weights [C_out, C_in, k, k] без
     смещения —
// то же, что torch::nn::functional::conv2d с параметрами stride, padding, dilation.
// Тензоры должны находиться на одном устройстве CUDA и иметь один тип (float, double,
     half или
// bfloat16); непрерывность в памяти не требуется: аксессоры учитывают шаги тензора.
// Кёрнел ставится в текущий поток PyTorch (at::cuda::getCurrentCUDAStream) и
     выполняется
// асинхронно; результат — новый непрерывный тензор [N, C_out, H_out, W_out].
template <long filter_size, long stride = 1, long padding = 0, long dilation = 1,
          long block_size = 16>
torch::Tensor conv2d(const torch::Tensor& input, const torch::Tensor& weights) {
    TORCH_CHECK(input.is_cuda() && weights.is_cuda(),
                "hsys::conv2d: tensors must be on a CUDA device, got ",
     input.device(), " and ",
                weights.device());
    TORCH_CHECK(input.device() == weights.device(),
                "hsys::conv2d: tensors must be on the same device, got ",
     input.device(),
                " and ", weights.device());
    TORCH_CHECK(input.scalar_type() == weights.scalar_type(),
                "hsys::conv2d: input and weights must have the same dtype, got ",
                input.scalar_type(), " and ", weights.scalar_type());
    TORCH_CHECK(input.dim() == 4, "hsys::conv2d: expected input [N, C_in, H_in, W_in],
     got ",
                input.sizes());
    TORCH_CHECK(weights.dim() == 4, "hsys::conv2d: expected weights [C_out, C_in, k,
     k], got ",
                weights.sizes());
    TORCH_CHECK(weights.size(1) == input.size(1), "hsys::conv2d: weights ",
     weights.sizes(),
                " expect ", weights.size(1), " input channels, got input ",
     input.sizes());
    TORCH_CHECK(weights.size(2) == filter_size && weights.size(3) == filter_size,
                "hsys::conv2d: kernel is compiled for a ", filter_size, "x",
     filter_size,
                " filter, got weights ", weights.sizes());


    const std::int64_t h_out =
        kernels::output_size(input.size(2), filter_size, stride, padding, dilation);
    const std::int64_t w_out =
        kernels::output_size(input.size(3), filter_size, stride, padding, dilation);
    TORCH_CHECK(h_out > 0 && w_out > 0, "hsys::conv2d: input ", input.sizes(),
                " is too small for a ", filter_size, "x", filter_size, " filter with
     padding ",
                padding, " and dilation ", dilation);


    const c10::cuda::CUDAGuard device_guard(input.device());


    torch::Tensor output =
        torch::empty({input.size(0), weights.size(0), h_out, w_out}, input.options());
    if (output.numel() == 0) {
        return output;  // Пустой пакет или C_out = 0: запуск с пустой сеткой
     недопустим.
    }


    const std::int64_t tiles_y = ceil_div(h_out, block_size);
    TORCH_CHECK(tiles_y <= kMaxGridYZ, "hsys::conv2d: output height ", h_out,
                " exceeds the grid limit");


    // Пары (изображение, группа каналов), которые не поместились в gridDim.z, кёрнел
```

```
    // обходит в цикле.
    const std::int64_t work_items =
        input.size(0) * ceil_div(weights.size(0), kernels::kOutChannelsPerBlock);

    const dim3 block(block_size, block_size);
    const dim3 grid(static_cast<unsigned int>(ceil_div(w_out, block_size)),
                    static_cast<unsigned int>(tiles_y),
                    static_cast<unsigned int>(std::min(work_items, kMaxGridYZ)));
    const cudaStream_t stream = at::cuda::getCurrentCUDAStream();

    AT_DISPATCH_FLOATING_TYPES_AND2(
        at::ScalarType::Half, at::ScalarType::BFloat16, input.scalar_type(),
     "hsys::conv2d", [&] {
           kernels::conv2d<scalar_t, filter_size, stride, padding, dilation,
     block_size>
                <<<grid, block, 0, stream>>>(
                    output.packed_accessor32<scalar_t, 4, torch::RestrictPtrTraits>(),
                    input.packed_accessor32<scalar_t, 4, torch::RestrictPtrTraits>(),
                    weights.packed_accessor32<scalar_t, 4,
     torch::RestrictPtrTraits>());
           C10_CUDA_KERNEL_LAUNCH_CHECK();
       });

    return output;
}

}  // namespace hsys::nn::functional
```

Листинг А.5 – Файл `tests/CMakeLists.txt`

```
add_executable(unit_tests
    src/conv2d_instances.cu
    src/test_conv2d.cpp
)

target_include_directories(unit_tests PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_link_libraries(unit_tests PRIVATE
    hsys::nn
    GTest::gtest_main
)

# Windows ищет DLL рядом с исполняемым файлом: копируем библиотеки LibTorch после
     сборки.
if(WIN32)
  file(GLOB TORCH_DLLS "${TORCH_INSTALL_PREFIX}/lib/*.dll")
  add_custom_command(TARGET unit_tests POST_BUILD
      COMMAND ${CMAKE_COMMAND} -E copy_if_different ${TORCH_DLLS}
     $<TARGET_FILE_DIR:unit_tests>
      VERBATIM
  )
endif()

# PRE_TEST: список тестов запрашивается при запуске ctest, когда DLL уже скопированы.
```

```
include(GoogleTest)
gtest_discover_tests(unit_tests DISCOVERY_MODE PRE_TEST DISCOVERY_TIMEOUT 120)
```

Листинг А.6 – Файл `tests/include/conv2d_instances.h`

```
#pragma once

#include <torch/types.h>

#include <string>

namespace hsys::tests {

// Параметры фильтра и блока, с которыми скомпилирован экземпляр
     hsys::nn::functional::conv2d.
struct FilterParams {
    long filter_size;
    long stride;
    long padding;
    long dilation;
    long block_size = 16;

    bool operator==(const FilterParams&) const = default;
};

// Имя для параметризованных тестов, например k3_s2_p1_d1 или k3_s1_p1_d1_b32.
std::string to_string(const FilterParams& params);

using Conv2dFn = torch::Tensor (*)(const torch::Tensor& input, const torch::Tensor&
     weights);

// Экземпляр conv2d для params; std::out_of_range, если он не скомпилирован. Параметры
     фильтра
// — аргументы шаблона, поэтому каждый проверяемый набор порождается в
     conv2d_instances.cu,
// а сами тесты компилируются обычным компилятором C++ без кода CUDA.
Conv2dFn conv2d_instance(const FilterParams& params);

}  // namespace hsys::tests
```

Листинг А.7 – Файл `tests/src/conv2d_instances.cu`

```
#include "conv2d_instances.h"

#include <array>
#include <stdexcept>
#include <utility>

#include "hsys/nn/functional/conv2d.cuh"

namespace hsys::tests {

namespace {

template <long filter_size, long stride, long padding, long dilation, long block_size
     = 16>
constexpr std::pair<FilterParams, Conv2dFn> instance() {
    return {{filter_size, stride, padding, dilation, block_size},
            &nn::functional::conv2d<filter_size, stride, padding, dilation,
     block_size>};
```

```
}

const std::array kInstances = {
    // Табл. 1 задания.
    instance<3, 2, 1, 1>(),
    instance<3, 1, 1, 1>(),
    instance<7, 2, 3, 1>(),
    instance<1, 2, 0, 1>(),
    instance<3, 1, 2, 2>(),
    instance<3, 2, 2, 2>(),
    // Другие размеры блока: тайл 8x8 и блок из 1024 нитей с наибольшим тайлом входа
     (69x69).
    instance<3, 1, 1, 1, 8>(),
    instance<3, 1, 1, 1, 32>(),
    instance<7, 2, 3, 1, 32>(),
    // Параметры вне таблицы: чётный фильтр, шаг больше фильтра, отступ больше
     половины
    // фильтра, фильтр без отступа (выход меньше входа).
    instance<2, 1, 0, 3>(),
    instance<1, 3, 0, 1>(),
    instance<5, 3, 0, 1>(),
    instance<3, 1, 3, 1>(),
};

}  // namespace

std::string to_string(const FilterParams& params) {
    std::string name = "k" + std::to_string(params.filter_size) + "_s" +
                       std::to_string(params.stride) + "_p" +
     std::to_string(params.padding) +
                       "_d" + std::to_string(params.dilation);
    if (params.block_size != 16) {
        name += "_b" + std::to_string(params.block_size);
    }
    return name;
}

Conv2dFn conv2d_instance(const FilterParams& params) {
    for (const auto& [instance_params, fn] : kInstances) {
        if (instance_params == params) {
            return fn;
        }
    }
    throw std::out_of_range("conv2d is not instantiated for " + to_string(params));
}

}  // namespace hsys::tests
```

Листинг А.8 – Файл `tests/src/test_conv2d.cpp`

```
#include <c10/cuda/CUDAGuard.h>
#include <c10/cuda/CUDAStream.h>
#include <gtest/gtest.h>
#include <torch/torch.h>

#include <cmath>
#include <cstdint>
#include <string>
#include <tuple>
#include <vector>

#include "conv2d_instances.h"

using hsys::tests::conv2d_instance;
```

```
using hsys::tests::FilterParams;

namespace F = torch::nn::functional;

namespace {

// Размер пакета в тестах табл. 1 (в задании не задан).
constexpr std::int64_t kBatchSize = 2;

struct Shape {
    std::int64_t batch;
    std::int64_t in_channels;
    std::int64_t height;
    std::int64_t width;
    std::int64_t out_channels;
};

std::string to_string(const Shape& shape) {
    return "n" + std::to_string(shape.batch) + "_" + std::to_string(shape.in_channels)
     + "x" +
           std::to_string(shape.height) + "x" + std::to_string(shape.width) + "_to" +
           std::to_string(shape.out_channels);
}

std::string to_string(torch::ScalarType dtype) {
    switch (dtype) {
        case torch::kHalf:
            return "Half";
        case torch::kBFloat16:
            return "BFloat16";
        case torch::kDouble:
            return "Double";
        case torch::kFloat:
        default:
            return "Float";
    }
}

// Табл. 1 задания: параметры фильтра x размеры изображений.
const std::vector<FilterParams> kTableFilters = {
    {3, 2, 1, 1}, {3, 1, 1, 1}, {7, 2, 3, 1}, {1, 2, 0, 1}, {3, 1, 2, 2}, {3, 2, 2,
     2},
};

const std::vector<Shape> kTableShapes = {
    {kBatchSize, 3, 244, 244, 64},   {kBatchSize, 64, 112, 112, 64},
    {kBatchSize, 64, 56, 56, 64},    {kBatchSize, 128, 28, 28, 128},
    {kBatchSize, 64, 56, 56, 128},   {kBatchSize, 128, 28, 28, 256},
    {kBatchSize, 256, 14, 14, 512},  {kBatchSize, 256, 14, 14, 256},
    {kBatchSize, 512, 7, 7, 512},
};

struct Tolerance {
    double rtol;
    double atol;
};

// Допуски задания: 1e-5 для float, 1e-2 для типов пониженной точности.
Tolerance tolerance(torch::ScalarType dtype) {
    switch (dtype) {
        case torch::kHalf:
        case torch::kBFloat16:
            return {1e-2, 1e-2};
        case torch::kDouble:
```

```
            return {1e-12, 1e-12};
        case torch::kFloat:
        default:
            return {1e-5, 1e-5};
    }
}

torch::TensorOptions cuda_options(torch::ScalarType dtype) {
    return torch::TensorOptions().device(torch::kCUDA).dtype(dtype);
}

// Равномерное распределение на [-bound, bound]; генерируется в float, затем
     приводится к dtype,
// поэтому тензоры разных типов содержат одни и те же значения с точностью до
     округления.
torch::Tensor uniform(torch::IntArrayRef sizes, double bound, torch::ScalarType dtype)
     {
    return ((torch::rand(sizes, cuda_options(torch::kFloat)) * 2 - 1) *
     bound).to(dtype);
}

// Вход из [-1, 1] и веса из [-1/sqrt(fan_in), 1/sqrt(fan_in)], как при инициализации
// torch::nn::Conv2d: выходы имеют порядок единицы при любом числе входных каналов.
std::pair<torch::Tensor, torch::Tensor> random_problem(const FilterParams& params,
                                                       const Shape& shape,
                                                       torch::ScalarType dtype) {
    const std::int64_t k = params.filter_size;
    const double bound = 1.0 / std::sqrt(static_cast<double>(shape.in_channels * k *
     k));
    return {uniform({shape.batch, shape.in_channels, shape.height, shape.width}, 1.0,
     dtype),
            uniform({shape.out_channels, shape.in_channels, k, k}, bound, dtype)};
}

F::Conv2dFuncOptions conv_options(const FilterParams& params) {
    return F::Conv2dFuncOptions()
        .stride(static_cast<std::int64_t>(params.stride))
        .padding(static_cast<std::int64_t>(params.padding))
        .dilation(static_cast<std::int64_t>(params.dilation));
}

// Эталон — torch::nn::functional::conv2d над теми же значениями в double, приведённый
     к типу
// входа: погрешность эталона не складывается с погрешностью проверяемого кёрнела, а
     TF32
// cuDNN (включён по умолчанию на Ampere) не участвует.
torch::Tensor reference(const torch::Tensor& input, const torch::Tensor& weights,
                        const FilterParams& params) {
    return F::conv2d(input.to(torch::kDouble), weights.to(torch::kDouble),
     conv_options(params))
        .to(input.scalar_type());
}

::testing::AssertionResult matches_reference(const torch::Tensor& input,
                                             const torch::Tensor& weights,
                                             const FilterParams& params) {
    const torch::Tensor actual = conv2d_instance(params)(input, weights);
    const torch::Tensor expected = reference(input, weights, params);

    if (actual.sizes() != expected.sizes()) {
        return ::testing::AssertionFailure()
               << "shape " << actual.sizes() << ", expected " << expected.sizes();
    }
    if (actual.scalar_type() != input.scalar_type()) {
        return ::testing::AssertionFailure() << "dtype " << actual.scalar_type();
    }
```

```
    const auto [rtol, atol] = tolerance(input.scalar_type());
    if (torch::allclose(actual, expected, rtol, atol)) {
        return ::testing::AssertionSuccess();
    }
    const double max_error =
        (actual.to(torch::kDouble) -
     expected.to(torch::kDouble)).abs().max().item<double>();
    return ::testing::AssertionFailure() << "max |actual - expected| = " << max_error;
}

// Тесты пропускаются, если устройство CUDA недоступно.
class CudaTest : public ::testing::Test {
protected:
    void SetUp() override {
        if (!torch::cuda::is_available()) {
            GTEST_SKIP() << "CUDA device is not available";
        }
        torch::manual_seed(42);
    }
};

template <class Param>
class CudaParamTest : public CudaTest, public ::testing::WithParamInterface<Param> {};

}  // namespace

// Основной тест задания: декартово произведение табл. 1 для float, half и bfloat16.
using TableParam = std::tuple<torch::ScalarType, FilterParams, Shape>;

class Conv2dTableTest : public CudaParamTest<TableParam> {};

TEST_P(Conv2dTableTest, MatchesTorchConv2d) {
    const auto& [dtype, params, shape] = GetParam();
    const auto [input, weights] = random_problem(params, shape, dtype);
    EXPECT_TRUE(matches_reference(input, weights, params));
}

INSTANTIATE_TEST_SUITE_P(Table1, Conv2dTableTest,
                         ::testing::Combine(::testing::Values(torch::kFloat,
     torch::kHalf,
                                                              torch::kBFloat16),
                                            ::testing::ValuesIn(kTableFilters),
                                            ::testing::ValuesIn(kTableShapes)),
                         [](const auto& info) {
                             return to_string(std::get<0>(info.param)) + "_" +
                                    hsys::tests::to_string(std::get<1>(info.param)) +
     "_" +
                                    to_string(std::get<2>(info.param));
                         });

// Граничные случаи: размеры, не кратные тайлу и группе каналов, выход 1x1, фильтр
     шире
// изображения, неквадратные изображения, размеры блока 8 и 32, параметры вне табл. 1.
namespace {

struct EdgeCase {
    std::string name;
    FilterParams params;
    Shape shape;
};

const std::vector<EdgeCase> kEdgeCases = {
    {"SinglePixel", {3, 1, 1, 1}, {1, 1, 1, 1, 1}},
```

```
    {"SingleChannel", {3, 2, 1, 1}, {2, 1, 31, 29, 1}},
    {"NotMultipleOfTile", {3, 1, 1, 1}, {3, 5, 17, 33, 5}},
    {"WideImage", {3, 1, 1, 1}, {1, 4, 5, 300, 13}},
    {"TallImage", {7, 2, 3, 1}, {1, 4, 300, 5, 9}},
    {"FilterWiderThanImage", {7, 2, 3, 1}, {2, 3, 1, 1, 9}},
    {"DilatedFilterWiderThanImage", {3, 1, 2, 2}, {2, 3, 1, 2, 7}},
    {"PointwiseOnSinglePixel", {1, 2, 0, 1}, {3, 8, 1, 1, 8}},
    {"PointwiseOddSize", {1, 2, 0, 1}, {2, 3, 15, 17, 9}},
    {"OutChannelsBelowGroup", {3, 2, 2, 2}, {2, 6, 23, 19, 3}},
    {"OutChannelsAboveGroup", {3, 2, 2, 2}, {2, 6, 23, 19, 9}},
    {"LargeBatch", {3, 1, 1, 1}, {17, 3, 9, 9, 4}},
    {"Block8", {3, 1, 1, 1, 8}, {2, 16, 33, 35, 24}},
    {"Block32", {3, 1, 1, 1, 32}, {2, 16, 33, 35, 24}},
    {"Block32LargestTile", {7, 2, 3, 1, 32}, {2, 16, 70, 66, 9}},
    {"EvenDilatedFilter", {2, 1, 0, 3}, {2, 3, 9, 4, 3}},
    {"StrideAboveFilter", {1, 3, 0, 1}, {2, 6, 23, 31, 17}},
    {"NoPadding", {5, 3, 0, 1}, {2, 6, 23, 31, 17}},
    {"PaddingAboveHalfFilter", {3, 1, 3, 1}, {2, 4, 11, 13, 10}},
};

}  // namespace

using EdgeParam = std::tuple<torch::ScalarType, EdgeCase>;

class Conv2dEdgeTest : public CudaParamTest<EdgeParam> {};

TEST_P(Conv2dEdgeTest, MatchesTorchConv2d) {
    const auto& [dtype, edge] = GetParam();
    const auto [input, weights] = random_problem(edge.params, edge.shape, dtype);
    EXPECT_TRUE(matches_reference(input, weights, edge.params));
}

INSTANTIATE_TEST_SUITE_P(EdgeCases, Conv2dEdgeTest,
                         ::testing::Combine(::testing::Values(torch::kFloat,
     torch::kDouble,
                                                              torch::kHalf),
                                            ::testing::ValuesIn(kEdgeCases)),
                         [](const auto& info) {
                             return to_string(std::get<0>(info.param)) + "_" +
                                    std::get<1>(info.param).name;
                         });

// Точное совпадение: целые значения из [-2, 2] в double суммируются без округления,
     поэтому
// любой пропущенный, лишний или сдвинутый элемент отступа меняет результат.
class Conv2dExactTest : public CudaParamTest<FilterParams> {};

TEST_P(Conv2dExactTest, IntegerValuesMatchExactly) {
    const FilterParams& params = GetParam();
    const std::int64_t k = params.filter_size;
    const auto options = cuda_options(torch::kDouble);
    const torch::Tensor input = torch::randint(-2, 3, {2, 5, 19, 23}, options);
    const torch::Tensor weights = torch::randint(-2, 3, {11, 5, k, k}, options);

    EXPECT_TRUE(torch::equal(conv2d_instance(params)(input, weights),
                             reference(input, weights, params)));
}

TEST_P(Conv2dExactTest, ImpulseReproducesFilter) {
    // Единичный импульс в центре входа: каждый выход, накрывающий импульс, равен
     одному
    // весу фильтра, остальные — нулю. Проверяет индексацию весов и тайла.
    const FilterParams& params = GetParam();
    const std::int64_t k = params.filter_size;
    const auto options = cuda_options(torch::kDouble);
```

```
    torch::Tensor input = torch::zeros({1, 1, 21, 21}, options);
    input.index_put_({0, 0, 10, 10}, 1.0);
    const torch::Tensor weights = torch::arange(1, k * k + 1, options).reshape({1, 1,
     k, k});

    EXPECT_TRUE(torch::equal(conv2d_instance(params)(input, weights),
                             reference(input, weights, params)));
}

INSTANTIATE_TEST_SUITE_P(Table1, Conv2dExactTest, ::testing::ValuesIn(kTableFilters),
                         [](const auto& info) { return
     hsys::tests::to_string(info.param); });

// Аксессоры учитывают шаги тензора: непрерывность входа и весов не требуется.
class Conv2dLayoutTest : public CudaTest {};

TEST_F(Conv2dLayoutTest, TransposedInput) {
    const FilterParams params{3, 2, 1, 1};
    const torch::Tensor input = uniform({2, 8, 27, 19}, 1.0,
     torch::kFloat).transpose(2, 3);
    const torch::Tensor weights = uniform({10, 8, 3, 3}, 0.1, torch::kFloat);
    ASSERT_FALSE(input.is_contiguous());
    EXPECT_TRUE(matches_reference(input, weights, params));
}

TEST_F(Conv2dLayoutTest, ChannelsLast) {
    const FilterParams params{3, 1, 1, 1};
    const torch::Tensor input = uniform({2, 8, 20, 20}, 1.0, torch::kFloat)
                                    .contiguous(torch::MemoryFormat::ChannelsLast);
    const torch::Tensor weights = uniform({16, 8, 3, 3}, 0.1, torch::kFloat)
                                      .contiguous(torch::MemoryFormat::ChannelsLast);
    EXPECT_TRUE(matches_reference(input, weights, params));
}

TEST_F(Conv2dLayoutTest, SlicedInputAndWeights) {
    // Каждый второй канал и столбец: шаги не совпадают с размерами.
    const FilterParams params{7, 2, 3, 1};
    const torch::Tensor input = uniform({2, 12, 30, 60}, 1.0, torch::kFloat)
                                    .slice(1, 0, 12, 2)
                                    .slice(3, 0, 60, 2);
    const torch::Tensor weights = uniform({20, 6, 7, 7}, 0.1, torch::kFloat).slice(0,
     0, 20, 2);
    EXPECT_TRUE(matches_reference(input, weights, params));
}

TEST_F(Conv2dLayoutTest, OutputIsContiguous) {
    const torch::Tensor input = uniform({1, 3, 8, 8}, 1.0, torch::kFloat).transpose(2,
     3);
    const torch::Tensor weights = uniform({4, 3, 3, 3}, 0.1, torch::kFloat);
    EXPECT_TRUE(conv2d_instance({3, 1, 1, 1})(input, weights).is_contiguous());
}

// Поведение функции запуска.
class Conv2dLaunchTest : public CudaTest {};

TEST_F(Conv2dLaunchTest, GridStrideOverBatch) {
    // N * ceil(C_out / 8) = 70000 > 65535: часть пар обходится циклом по blockIdx.z.
    const FilterParams params{1, 2, 0, 1};
    const torch::Tensor input = uniform({70000, 1, 1, 1}, 1.0, torch::kFloat);
    const torch::Tensor weights = uniform({1, 1, 1, 1}, 1.0, torch::kFloat);
    EXPECT_TRUE(matches_reference(input, weights, params));
}

TEST_F(Conv2dLaunchTest, EmptyBatch) {
```

```
    const torch::Tensor input = torch::empty({0, 3, 8, 8},
     cuda_options(torch::kFloat));
    const torch::Tensor weights = uniform({4, 3, 3, 3}, 0.1, torch::kFloat);


    const torch::Tensor output = conv2d_instance({3, 1, 1, 1})(input, weights);
    EXPECT_EQ(output.sizes().vec(), (std::vector<std::int64_t>{0, 4, 8, 8}));
}


TEST_F(Conv2dLaunchTest, UsesCurrentStream) {
    const FilterParams params{3, 1, 1, 1};
    const torch::Tensor input = uniform({2, 16, 64, 64}, 1.0, torch::kFloat);
    const torch::Tensor weights = uniform({32, 16, 3, 3}, 0.1, torch::kFloat);
    const torch::Tensor expected = reference(input, weights, params);


    const c10::cuda::CUDAStream stream = c10::cuda::getStreamFromPool();
    torch::Tensor actual;
    {
        const c10::cuda::CUDAStreamGuard guard(stream);
        actual = conv2d_instance(params)(input, weights);
    }
    stream.synchronize();
    EXPECT_TRUE(torch::allclose(actual, expected, 1e-5, 1e-5));
}


TEST_F(Conv2dLaunchTest, DeterministicAndInputsUnchanged) {
    const FilterParams params{3, 2, 2, 2};
    const torch::Tensor input = uniform({2, 32, 28, 28}, 1.0, torch::kFloat);
    const torch::Tensor weights = uniform({24, 32, 3, 3}, 0.1, torch::kFloat);
    const torch::Tensor input_copy = input.clone();
    const torch::Tensor weights_copy = weights.clone();


    const torch::Tensor first = conv2d_instance(params)(input, weights);
    for (int i = 0; i < 5; ++i) {
        EXPECT_TRUE(torch::equal(conv2d_instance(params)(input, weights), first));
    }
    EXPECT_TRUE(torch::equal(input, input_copy));
    EXPECT_TRUE(torch::equal(weights, weights_copy));
}


TEST_F(Conv2dLaunchTest, OutputSizeMatchesFormula) {
    // Формулы (3)-(4) против размеров, которые возвращает PyTorch.
    for (const FilterParams& params : kTableFilters) {
        for (const std::int64_t size : {1, 2, 3, 7, 8, 14, 15, 28, 56, 112, 244}) {
            const auto options = cuda_options(torch::kFloat);
            const torch::Tensor input = torch::zeros({1, 1, size, size + 1}, options);
            const torch::Tensor weights =
                torch::zeros({1, 1, params.filter_size, params.filter_size}, options);
            EXPECT_EQ(conv2d_instance(params)(input, weights).sizes(),
                      F::conv2d(input, weights, conv_options(params)).sizes())
                << hsys::tests::to_string(params) << ", size " << size;
        }
    }
}


// Некорректные аргументы отклоняются TORCH_CHECK (c10::Error) до запуска кёрнела.
class Conv2dErrorTest : public CudaTest {
protected:
    void SetUp() override {
        CudaTest::SetUp();
        if (IsSkipped()) {
            return;
        }
        input_ = uniform({1, 4, 8, 8}, 1.0, torch::kFloat);
        weights_ = uniform({2, 4, 3, 3}, 0.1, torch::kFloat);
    }
```

```
    const hsys::tests::Conv2dFn conv3x3_ = conv2d_instance({3, 1, 1, 1});
    torch::Tensor input_;
    torch::Tensor weights_;
};

TEST_F(Conv2dErrorTest, CpuTensors) {
    EXPECT_THROW(conv3x3_(input_.cpu(), weights_.cpu()), c10::Error);
    EXPECT_THROW(conv3x3_(input_, weights_.cpu()), c10::Error);
}

TEST_F(Conv2dErrorTest, DtypeMismatch) {
    EXPECT_THROW(conv3x3_(input_, weights_.to(torch::kDouble)), c10::Error);
}

TEST_F(Conv2dErrorTest, UnsupportedDtype) {
    EXPECT_THROW(conv3x3_(input_.to(torch::kInt), weights_.to(torch::kInt)),
     c10::Error);
}

TEST_F(Conv2dErrorTest, WrongRank) {
    EXPECT_THROW(conv3x3_(input_[0], weights_), c10::Error);
    EXPECT_THROW(conv3x3_(input_, weights_[0]), c10::Error);
}

TEST_F(Conv2dErrorTest, InputChannelsMismatch) {
    EXPECT_THROW(conv3x3_(input_.slice(1, 0, 3), weights_), c10::Error);
}

TEST_F(Conv2dErrorTest, FilterSizeMismatch) {
    EXPECT_THROW(conv3x3_(input_, uniform({2, 4, 5, 5}, 0.1, torch::kFloat)),
     c10::Error);
    EXPECT_THROW(conv3x3_(input_, uniform({2, 4, 3, 1}, 0.1, torch::kFloat)),
     c10::Error);
}

TEST_F(Conv2dErrorTest, InputTooSmall) {
    // Фильтр 2x2 с расширением 3 охватывает 4 пикселя: вход 3x3 без отступа для него
     мал.
    // Числитель формулы (3) равен -1, усечение деления к нулю дало бы размер 1.
    const torch::Tensor input = uniform({1, 1, 3, 3}, 1.0, torch::kFloat);
    const torch::Tensor weights = uniform({1, 1, 2, 2}, 1.0, torch::kFloat);
    EXPECT_THROW(conv2d_instance({2, 1, 0, 3})(input, weights), c10::Error);
    EXPECT_THROW(F::conv2d(input, weights, conv_options({2, 1, 0, 3})), c10::Error);
}

TEST(Conv2dInstancesTest, MissingInstanceThrows) {
    EXPECT_THROW(conv2d_instance({11, 1, 0, 1}), std::out_of_range);
}
```